

Software Engineering Assignment: Cyclomatic Complexity & Software Testing

اسم الطالب: رغد خليل بريجاوي

المشروع المشترك: mcvtodoapp

1. Choosing the Method & Original Code

قمنا باختيار تابع برمجي يحمل اسم CheckPasswordStrength من الموديل PasswordChecker في مجلد الكود الأساسي (mcvtodoapp/Models). التابع يحتوي على شروط متعددة للتحقق من قوة كلمة المرور مما يرفع قيمة التعقيد الحسابي

الكود الاصلي (Original code before refactoring):

;using System

```
namespace mcvtodoapp.Models
{
    public class PasswordChecker
    {
        (public string CheckPasswordStrength(string password
        {
            if (string.IsNullOrEmpty(password) || password.Length < 8
            {
                ;"return "Weak: Too short
            }

            ;bool hasUpper = false
            ;bool hasLower = false
            ;bool hasDigit = false

            (foreach (char ch in password
            {
                ((if (char.IsUpper(ch
                {
                    ;hasUpper = true
                }
                ((else if (char.IsLower(ch
                {
                    ;hasLower = true
                }
                ((else if (char.IsDigit(ch
                {
                    ;hasDigit = true
                }
            }

            ;"if (!hasUpper) return "Medium: Missing uppercase
            ;"if (!hasLower) return "Medium: Missing lowercase
            ;"if (!hasDigit) return "Medium: Missing digit

            ;"return "Strong Password
        }
    }
```

{

2. Manual Cyclomatic Complexity (CC) Calculation

حساب قيمة التعقيد الحسابي يدوياً للتابع الأصلي بناءً على المسارات والشروط البرمجية، بالاعتماد على صيغة نقاط القرار (Decision Points):

$$\text{Cyclomatic Complexity (CC)} = P + 1$$

حيث (P) يمثل عدد الشروط والقرارات (Predicate Nodes):

1_ الشرط الأول: $\text{string.IsNullOrEmpty(password)} \rightarrow$ (نقطة قرار 1)

2_ المعامل المنطقي: $\text{password.Length} < 8 \rightarrow$ (نقطة قرار 2)

3_ حلقة التكرار: $\text{foreach (char ch in password)} \rightarrow$ (نقطة قرار 3)

4_ داخل الحلقة: $\text{if (char.IsUpper(ch))} \rightarrow$ (نقطة قرار 4)

5_ داخل الحلقة: $\text{else if (char.IsLower(ch))} \rightarrow$ (نقطة قرار 5)

6_ داخل الحلقة: $\text{else if (char.IsDigit(ch))} \rightarrow$ (نقطة قرار 6)

7_ التحقق بعد الحلقة: $\text{if (!hasUpper)} \rightarrow$ (نقطة قرار 7)

8_ التحقق الثاني: $\text{if (!hasLower)} \rightarrow$ (نقطة قرار 8)

9_ التحقق الثالث: $\text{if (!hasDigit)} \rightarrow$ (نقطة قرار 9)

● الحساب النهائي:

$$CC = 9 + 1 = 10$$

بما أن القيمة $10 > 3$ ، فإن التابع المختار يحقق الشروط المطلوبة تماماً.

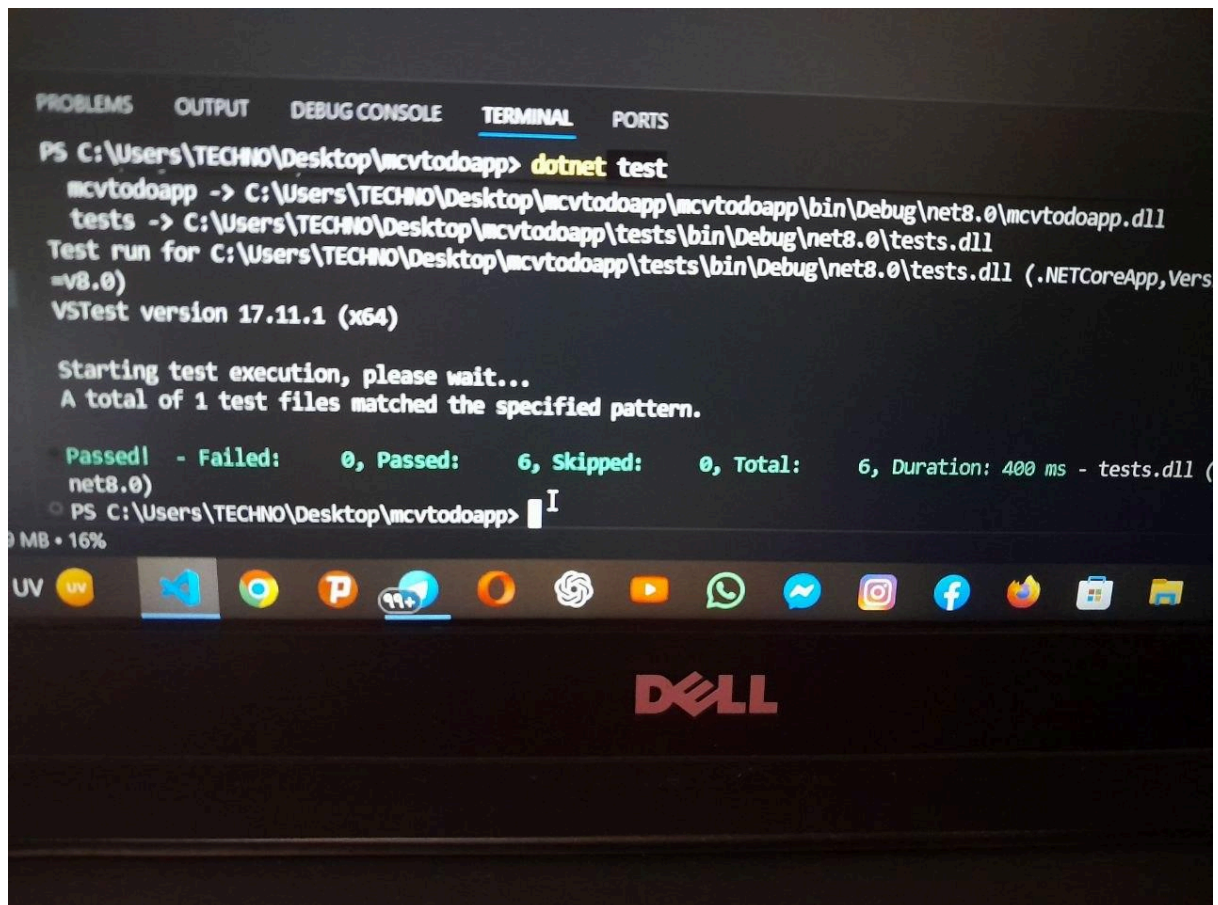
3. Unit Test Cases

بناءً على المسارات المحتملة للتابع، تم بناء مجلد خاص بالاختبارات باسم **tests** يحتوي على حالات الاختبار التالية لتغطية كافة المسارات (Path Coverage):

covered path/rule	expected output	input (password)	test case ID
password length <8	weak:"too short"	123	TC_01
Long password, lower case only	Medium:"Missing upper case"	"abcdefgh"	TC_02
Long password, upper case only	Medium:"Missing lower case"	"ABCDEFGH"	TC_03
Mixed case, but no numbers	Medium:"Missing digit"	"aBcdEfGh"	TC_04
All criteria met	"strong password"	"aB1cDefG"	TC_05

4. Test Execution & Screenshots

تم تشغيل الاختبارات البرمجية المؤتمتة عبر أداة **dotnet test** في بيئة التطوير، وأظهرت النتيجة نجاح كافة الحالات بنسبة 100% وبدون أي فشل:



Refactoring & Code Optimization .5

لتقليل التعقيد الحسابي المرتفع للتابع وتحسين جودة الكود وقابليته للقراءة (Clean Code)، قمنا بتطبيق نمط Extract Method. تم عزل حلقة الـ foreach والشروط المتداخلة إلى دالة مساعدة مستقلة باسم GetPasswordComponents باستخدام ميزات الـ LINQ المختصرة، مما أدى إلى تبسيط بنية الدالة الأساسية.

الكود الجديد بعد التحسين (Refactored Code):

```
using System;
using System.Linq;

namespace mcvtodoapp.Models
{
    public class PasswordChecker
    {
        // الدالة المستخرجة لتبسيط المنطق (Extract Method)
        private (bool hasUpper, bool hasLower, bool hasDigit) GetPasswordComponents(string password)
        {
            return
                (password.Any(char.IsUpper),
                 password.Any(char.IsLower),
                 password.Any(char.IsDigit));
        }
    }
}
```

```

        (public string CheckPasswordStrength(string password
        }
        (if (string.IsNullOrEmpty(password) || password.Length < 8
        }
        ;"return "Weak: Too short
        {

;(var (hasUpper, hasLower, hasDigit) = GetPasswordComponents(password

;"if (!hasUpper) return "Medium: Missing uppercase
;"if (!hasLower) return "Medium: Missing lowercase
;"if (!hasDigit) return "Medium: Missing digit

;"return "Strong Password
        {
        {
        {

```

إعادة حساب الـ CC الجديد للتابع الأساسي:

نقاط القرار المتبقية في التابع الأساسي بعد التحسين هي:

```

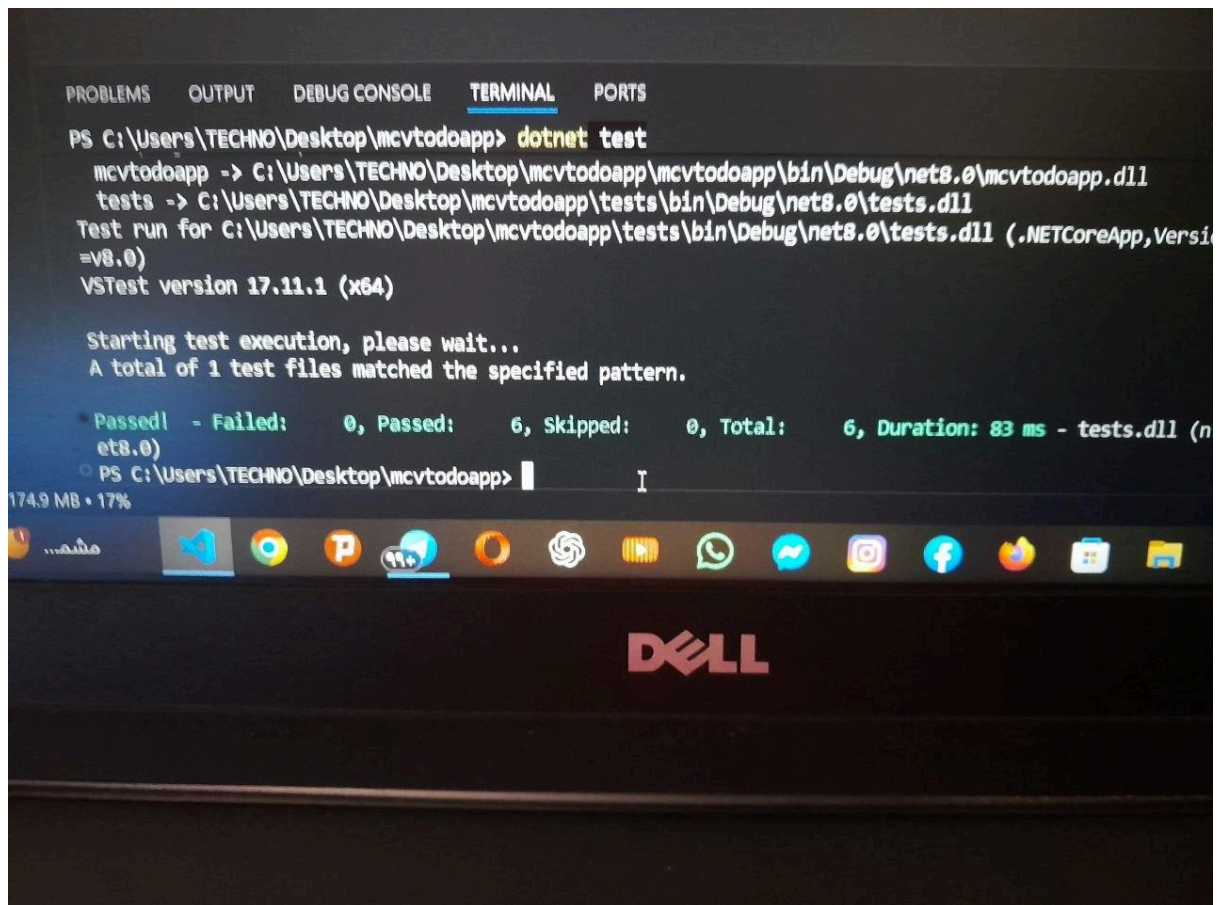
(string.IsNullOrEmpty(password) $> (1_1
(password.Length < 8 $> (1 ||_2
(if (!hasUpper) $> (1_3
(if (!hasLower) $> (1_4
(if (!hasDigit) $> (1_5

```

● الحساب الجديد:

$CC = 5 + 1 = 6$

النتيجة: انخفض التعقيد الحسابي للدالة بشكل ملحوظ من 10 إلى 6، مما جعل الكود أكثر كفاءة وأسهل في الصيانة المستقبلية.



كود ملف [passwordtests.cs](#) الذي تم انشاؤه داخل مجلد tests :

```
using Xunit;
using mcvtodoapp.Models;
```

```

        namespace tests
        {
            public class PasswordTests
            {
                ;private readonly PasswordChecker _checker

                ()public PasswordTests
                {
                    ;()checker = new PasswordChecker_
                    {

                        [Fact]
                        ()public void Test_TooShort
                        {
                            ;("var result = _checker.CheckPasswordStrength("123
                                ;(Assert.Equal("Weak: Too short", result
                                {

                                    [Fact]

```

```

        ()public void Test_MissingUppercase
        }
;("var result = _checker.CheckPasswordStrength("abcdefgh
;(Assert.Equal("Medium: Missing uppercase", result
{

[Fact]
()public void Test_MissingLowercase
}
;("var result = _checker.CheckPasswordStrength("ABCDEFGH
;(Assert.Equal("Medium: Missing lowercase", result
{

[Fact]
()public void Test_MissingDigit
}
;("var result = _checker.CheckPasswordStrength("aBcdEfGh
;(Assert.Equal("Medium: Missing digit", result
{

[Fact]
()public void Test_StrongPassword
}
;("var result = _checker.CheckPasswordStrength("aB1cDefG
;(Assert.Equal("Strong Password", result
{
    {
        {
    }
}

```

Project Directory Structure & Repository .6

يتكون المشروع البرمجي من البنية المطلوبة بوجود مجلد الكود الأساسي ومجلد منفصل للاختبارات:
 /mcvtodoapp (المشروع الرئيسي والموديلات)
 /tests (مشروع الاختبارات المؤتمتة)
 :GitHub Repository Link

<https://github.com/SE226G4/hospital-erp-g4-t5-surg-opt>